

This example creates a random walk with 50,000 points and plots each point at size `s=1`. The resulting walk is wispy and cloudlike, as shown in Figure 15-11. We’ve created a piece of art from a simple scatter plot!

Experiment with this code to see how much you can increase the number of points in a walk before your system starts to slow down significantly or the plot loses its visual appeal.

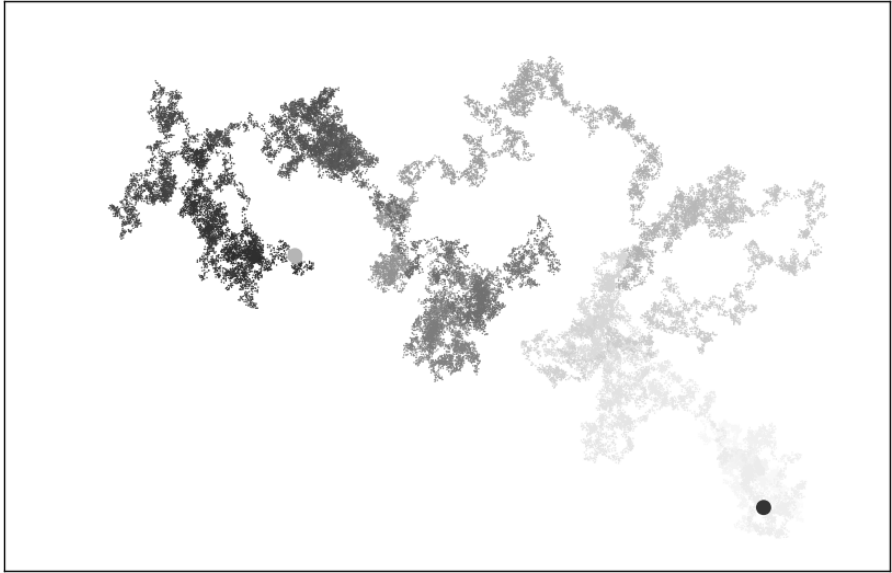


Figure 15-11: A walk with 50,000 points

Altering the Size to Fill the Screen

A visualization is much more effective at communicating patterns in data if it fits nicely on the screen. To make the plotting window better fit your screen, you can adjust the size of Matplotlib’s output. This is done in the `subplots()` call:

```
fig, ax = plt.subplots(figsize=(15, 9))
```

When creating a plot, you can pass `subplots()` a `figsize` argument, which sets the size of the figure. The `figsize` parameter takes a tuple that tells Matplotlib the dimensions of the plotting window in inches.

Matplotlib assumes your screen resolution is 100 pixels per inch; if this code doesn’t give you an accurate plot size, adjust the numbers as necessary. Or, if you know your system’s resolution, you can pass `subplots()` the resolution using the `dpi` parameter:

```
fig, ax = plt.subplots(figsize=(10, 6), dpi=128)
```

This should help make the most efficient use of the space available on your screen.